We have oversimplified the inverted index here, but that's all right, as our aim is to gain a basic understanding of the data structure. If we now search for *hello mate*, for example, both documents 1 and 2 are returned, but document 2 probably has a higher relevancy score than document 1 because the contents of document 2 match the query.

While an inverted index is optimized for faster information retrieval, it adds complexity to analysis and requires more space. The inverted index grows as indexing activity increases, thus consuming computing resources and heap space. We will not work directly with the data structure, but understanding the basics helps.

The inverted index also helps deduce relevancy scores; it provides the frequency of terms, which is one of the ingredients in calculating the relevancy score. We've used the term *relevancy* for a while, and it is time to understand what it is and what algorithms a search engine like Elasticsearch uses to fetch relevant results for the user. We discuss concepts related to relevancy in the next section.

3.4 Relevancy

Modern search engines not only return results based on query criteria but also analyze and return the most *relevant* results. If you are a developer or DevOps engineer, you have most likely used Stack Overflow to search for answers to technical problems. Searching on Stack Overflow never disappoints me (at least, in most cases). The results I get for a query are close to what I'm looking for. The results are sorted in a decent order, from highly relevant results at the top to the least relevant ones at the bottom. It would be highly unlikely for the results of a query not to satisfy your requirements—but you might not return to Stack Overflow if the results were all over the place.

3.4.1 Relevancy scores

Stack Overflow applies a set of relevancy algorithms to search and sort the results it returns to the user. Similarly, Elasticsearch returns results for full-text queries sorted, by a *relevancy score*.

Relevancy is a positive floating-point number that determines the ranking of search results. Elasticsearch uses the Best Match 25 (BM25) relevancy algorithm by default to score return results so the client can expect relevant results. It's an advanced version of the previously used term-frequency/inverse document frequency (TF/IDF) similarity algorithm (see the next section for more about relevancy algorithms).

If we are searching for, say, *Java* in the title of a book, a document containing more than one occurrence of the word *Java* in the title is more relevant than documents where the title has one or no occurrence. Look at the example results in figure 3.19, retrieved by Elasticsearch for a query searching for the keyword *Java* in a title (the full example is provided in the book's repository).